

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Soln:

1)

```
DECLARE
BEGIN
    FOR c IN (
        SELECT customer_id, age, interest_rate FROM Customers)
    LOOP
        IF c.age > 60 THEN
            UPDATE Customers SET interest_rate = interest_rate - 1 WHERE customer_id = c.customer_id;
        END IF;
    END LOOP;
END;
/
```

2)

```
DECLARE
BEGIN
    FOR c IN (SELECT customer_id, balance FROM Customers)
```

```

LOOP
    IF c.balance > 10000 THEN
        UPDATE Customers SET IsVIP = 'TRUE' WHERE customer_id = c.customer_id;
    END IF;
END LOOP;
END;
/

3)

DECLARE
BEGIN
    FOR I IN ( SELECT customer_name, due_date FROM Loans WHERE due_date <= SYSDATE + 30) LOOP
        DBMS_OUTPUT.PUT_LINE( 'Reminder: Loan due for ' || I.customer_name );
    END LOOP;
END;
/

```

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

SOLN:

1) CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest

IS

BEGIN

 UPDATE Accounts

 SET balance = balance + (balance * 0.01)

 WHERE account_type = 'SAVINGS';

END;

/

2)

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus

(p_department IN VARCHAR2,

 p_bonus IN NUMBER)

IS

BEGIN

 UPDATE Employees

 SET salary = salary +

 (salary * p_bonus / 100)

 WHERE department = p_department;

END;

/

3)

CREATE OR REPLACE PROCEDURE TransferFunds

(p_from_account IN NUMBER, p_to_account IN NUMBER, p_amount IN NUMBER)

IS

 v_balance NUMBER;

BEGIN

 SELECT balance

 INTO v_balance

 FROM Accounts

 WHERE account_id = p_from_account;

```
IF v_balance >= p_amount THEN

    UPDATE Accounts

    SET balance = balance - p_amount

    WHERE account_id = p_from_account;

    UPDATE Accounts

    SET balance = balance + p_amount

    WHERE account_id = p_to_account;

    DBMS_OUTPUT.PUT_LINE(

        'Transfer Successful'

    );

ELSE

    DBMS_OUTPUT.PUT_LINE(

        'Insufficient Balance'

    );

END IF;

END;

/
```